Study notes

Subject: CNN Foundations: Convolution/ Spatial Arithmetic / Receptive Field

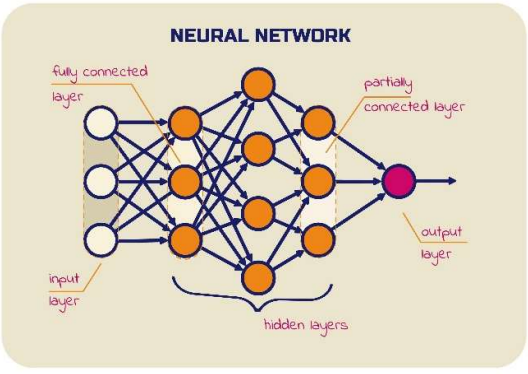
Author: Michael Mizzi

Date: 09th May 2026

Table of Contents

Background

Artificial Neural Networks (ANNs), standard multilayer feedforward networks, are systems where each unit in a hidden layer is partially (or fully) connected to every single unit in the previous layer. ANNs have journeyed from the humble, single-layer perceptrons of the 1950s to today's massive deep-learning architectures, finally hitting their stride once backpropagation met the raw horsepower of modern GPUs. Their importance lies in shifting machine learning from strict, hand-crafted rules to a paradigm where models autonomously learn complex, hierarchical patterns directly from raw data.



Applying ANNs to image processing implies that preset buckets of pixels are connected to each node and is subsequently memorized by the models through learning. Training phases are used to set weights at each node level of the hidden layers. As an analogy, we will consider an object detection task, ie; locating a car in an image. An ANN behaves like a human being scanning and memorizing a whole range of images containing cars of various brands, shapes and surrounding backgrounds. Once memorized the human is presented with a new image and will determine if a car like those memorized is located at any local region of the new image. While powerful, this methodology requires deep and complex memorizing and learning skills as new images are studied at pixel level.

We conclude that while ANNs can learn complex, high-dimensional mappings, they pose three significant limitations for computer vision compared to **Convolutional Neural Networks (CNNs)**:

1. **Parameter Explosion:** Large images contain hundreds or thousands of pixels. A fully connected layer with just 100 hidden units processing a medium-sized image would require tens of thousands of weights, which increases system capacity and necessitates massive training sets to prevent overfitting.
2. **Ignoring Spatial Topology:** ANNs treat an image as an unordered collection of variables, entirely ignoring its 2D structure. In contrast, images have strong local correlations where nearby pixels are highly related, a property ANNs do not exploit.
3. **Lack of Invariance:** ANNs have no built-in mechanism to handle translations or local distortions. If an object shifts slightly, a fully connected network might fail to recognize it unless it has specifically been trained on a similar pattern at that exact new location.

Throughout the next sections, we deep-dive into the very basics of CNNs which explain the concepts how the human scanning cars in an image shifts the objective from learning all the vast range of cars and environments to a smaller focus (receptive field). This focused process searches only for specific elements through a template, (kernel), that resemble the key features of a car. By ignoring the irrelevant parts of the environments surrounding cars and focusing only on local features and therefore requires less memory and improved structured learning skills.

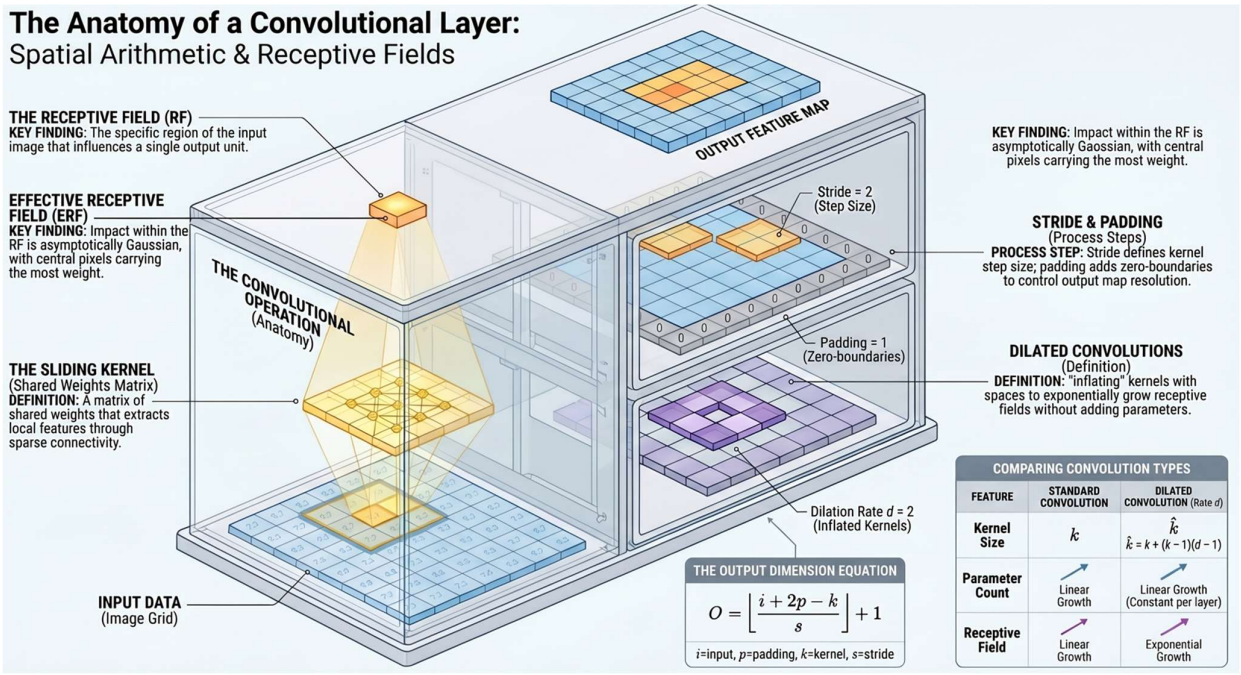
*The building blocks of a Convolutional Neural Network (CNN) follow a structured pipeline designed to transform raw input into high-level intelligence. It begins with **raw data**, typically pixels in an image, which is processed by a **convolutional layer** where small windows called **kernels** slide across the input to extract local features like edges, textures, and patterns. These feature maps are then "filtered" through an **activation function** (most commonly ReLU), which introduces non-linearity by deciding which signals are important enough to pass forward. To manage computational complexity and ensure the network is robust to small changes in the image, **pooling** layers downsample the data, effectively shrinking the spatial dimensions while preserving the most critical information.*

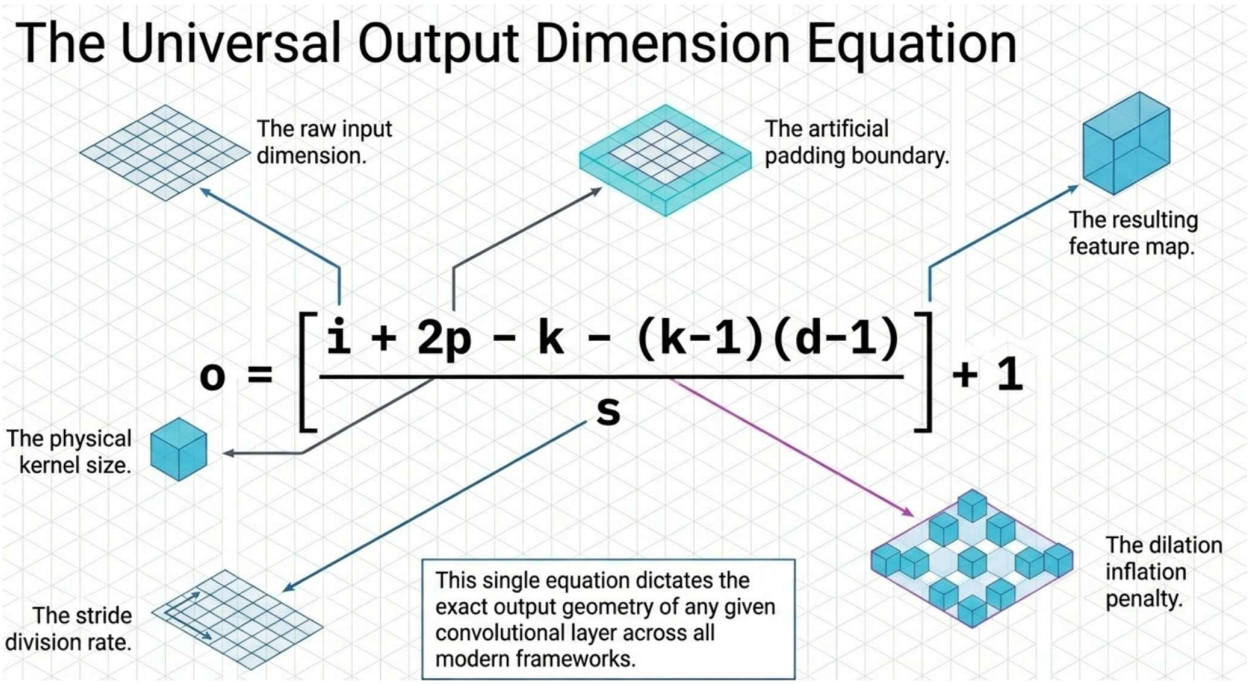
*After the spatial features are refined through multiple stages of convolution and pooling, the 2D feature maps are "flattened" into a 1D vector to enter the **fully connected ANN** (Artificial Neural Network). This part of the architecture functions as the global decision-maker, taking the high-level features identified in the previous steps and learning how they combine to represent complex objects. The final **output** layer uses specialized mathematical functions to translate these internal signals into a human-readable result. Depending on the specific task, this could be a **classification** (e.g., "this is a car"), **detection** (identifying the object's location with a bounding box), or **segmentation** (mapping the object pixel by pixel).*

Quick Reference

- **ANN**: 'Artificial Neural Networks' – Fully/partial connected networks where every unit in a hidden layer depends on the entire input.
- **CNN**: 'Convolution Neural Networks' - Specialized architecture using local connectivity and shared weights to exploit spatial topology in data.
- **KERNEL**: A sliding matrix/array of weights that computes element-wise products and summations across input maps.
- **PADDING**: Zeros added to input borders to control output spatial dimensions and maintain feature resolution.
- **STRIDE**: The step size between consecutive kernel positions during traversal, acting as a form of subsampling.
- **RECEPTIVE FIELD**: The specific spatial region in the input image that influences a particular output feature.
- **ERF**: 'Effective Receptive Field' - The central input area with non-negligible impact, which typically follows a Gaussian distribution.
- **OUTPUT EQUATION**: The formula that is used to calculate resulting feature map dimensions based on the input image size, kernel size, padding, stride and dilation parameters.

Visual summary of Convolution basics





Convolution

A discrete convolution is a linear transformation designed to preserve the spatial ordering of input data while exploiting its implicit topological structure. It is fundamentally defined by two properties: sparse connectivity, where only a few input units contribute to a specific output, and shared weights, where the same parameters are reused across multiple locations in the input.

The basic computation method follows these steps:

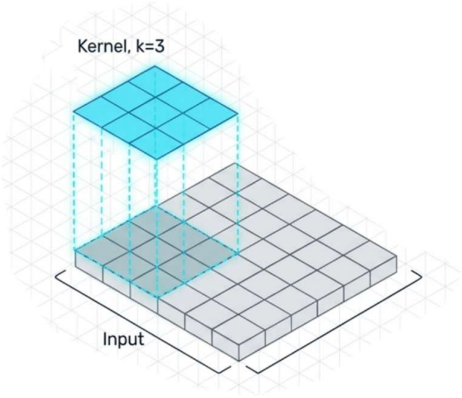
- **Sliding Window Mechanism:** A small array of weights, known as a kernel or filter, slides across the input feature map.
- **Element-wise Multiplication:** At every position the kernel occupies, each of its weights is multiplied by the corresponding input unit it currently overlaps.
- **Summation:** All individual products from that specific kernel placement are summed together to produce a single numerical value for that coordinate in the output feature map.
- **Multi-Channel Integration:** In modern architectures with multiple input channels (such as the Red, Green, and Blue channels of a color image), a distinct kernel is applied to each channel independently. The results from all channels are then summed element-wise to generate the final value for that spatial location in the output.
- **Bias and Activation:** This resulting sum is typically followed by the addition of a trainable bias and then processed through a non-linear activation function (or "squashing function"), such as a ReLU or sigmoid, to produce the final state of the output unit.

From a mathematical perspective, this entire operation can be viewed as a **sparse matrix multiplication** if the input and output are unrolled into vectors, where the non-zero elements of the matrix represent the kernel weights.

Hyperparameters

Kernel

In Convolutional Neural Networks (CNNs), a kernel (also referred to as a filter) is a small multidimensional array of trainable weights that traverses an input feature map to perform a linear transformation. Unlike fully connected networks where every output unit depends on the entire input, kernels enforce sparse connectivity—meaning an output unit only depends on a small, local region of the input—and shared weights, where the same set of weights is applied across every location in the input.



Common types of kernels used in CNN architectures:

1. Standard (Discrete) Convolutional Kernels

This is the fundamental kernel type used for feature extraction. It slides across the input with a specific stride (step size), performing element-wise multiplications and summations at each position to produce an output feature map.

- **Dimensions:** While often represented as 2D for simplicity, kernels are effectively 3D in multi-channel networks (e.g., RGB images), with a distinct 2D slice of weights convolving each input channel.
- **Purpose:** They extract elementary visual features like oriented edges, corners, or endpoints.

2. Dilated (Atrous) Kernels

Dilated kernels "inflate" the standard kernel by inserting spaces (zeros) between its elements, controlled by a hyperparameter called the dilation rate (*d*).

- **Effective Size:** A kernel of size '*k*' with dilation '*d*' has an expanded spatial span of '$d(k - 1) + 1$'.
- **Advantage:** They allow for an exponential expansion of the receptive field without increasing the number of trainable parameters or losing spatial resolution. They are particularly useful for dense prediction tasks like semantic segmentation.

3. Transposed Convolutional Kernels (Fractionally Strided)

Often used in the decoding layers of autoencoders, these kernels work by swapping the forward and backward passes of a standard convolution.

- **Upsampling:** They map from a smaller-dimensional space to a larger one while maintaining a connectivity pattern compatible with a standard convolution.
- **Mechanism:** Conceptually, this involves inserting zeros between input units (stretching the input) to allow the kernel to move at a slower, "fractional" pace.

4. Separable Kernels (Depth-wise and Spatial)

Separable convolutions decouple the convolution process to improve efficiency.

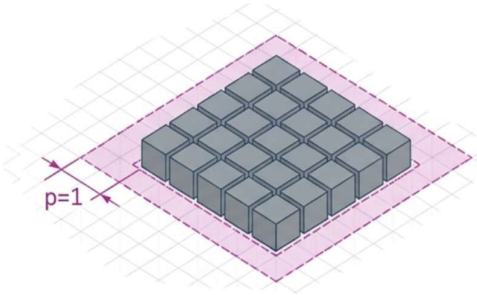
- **Depth-wise Separable:** These apply a single kernel to each input channel independently, rather than convolving across all channels simultaneously.
- **Efficiency:** This approach significantly reduces the computational footprint (FLOPs) and is a core component of modern, compact architectures like MobileNets.

5. Pooling Kernels

While not containing trainable weights like convolutional kernels, pooling kernels function as sliding windows that summarize local subregions.

- **Max Pooling:** Outputs the maximum value from the window to provide invariance to small translations.
- **Average Pooling:** Computes the average of the inputs in the window to reduce spatial resolution and sensitivity to distortions.

Padding



In Convolutional Neural Networks (CNNs), padding is the process of adding extra elements—most commonly zeros—to the borders of an input feature map before a convolution operation is performed. This technique is used to control the spatial dimensions of the output and ensure that features at the edges of the input are processed appropriately.

Purpose of Padding

- **Controlling Output Size:** Without padding, the spatial dimensions of a feature map naturally shrink after each convolutional layer because the kernel cannot centered on the very edge pixels without falling off the boundary.
- **Border Feature Preservation:** Padding allows the kernel to visit pixels at the extreme edges of the image as many times as central pixels, preventing the loss of information at the borders.
- **Resolution Maintenance:** In deep networks, padding is essential to prevent the feature maps from eventually disappearing due to successive reductions in size.

Types of Padding Used in CNNs

Based on the sources, there are several distinct types and configurations of padding:

1. **Zero Padding:** The most fundamental type, where a specific number of zeros (*p*) are concatenated at the beginning and end of an axis. This increases the effective input size from *i* to *i* + *2p* for the purpose of arithmetic calculations.
2. **Half (Same) Padding:** This is a specific configuration where the padding is set to $\boldsymbol{p = \lfloor k/2 \rfloor}$ (where *k* is the kernel size). When used with a stride of 1, this ensures the output size is

identical to the input size ($o = i$). It is widely used when maintaining the spatial resolution of feature maps is desirable.

3. **Full Padding:** In this setting, padding is set to $\boldsymbol{p = k - 1}$. This allows the kernel to visit every possible partial or complete overlap with the input feature map, resulting in an output larger than the input ($o = i + k - 1$).
4. **Reflection Padding:** Often used in dense prediction tasks like semantic segmentation (e.g., in the "front-end" of dilated convolution modules), this type fills the buffer zone by reflecting the image about each edge rather than using zeros. This helps avoid artifacts that can occur at the borders when using zero padding.
5. **Negative Padding (Cropping):** While padding usually implies adding to the input, a general mathematical definition also allows for negative padding, which is interpreted as cropping the edges of the input feature map.
6. **Valid (No) Padding:** This refers to the setting where $p=0$. The kernel only moves within the bounds of the original input, and any pixel that cannot be fully covered by the kernel at the end of a stride is ignored.

Padding in Convolution Arithmetic

In convolution, alongside the kernel size and stride steps, padding is a central variable in the standard output dimension equation:

$$o = \left\lfloor \frac{i + 2p - k}{s} \right\rfloor + 1$$

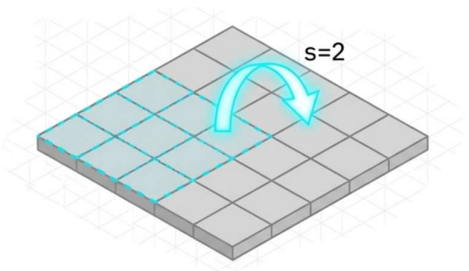
By adjusting p, designers can precisely determine the resulting feature map size (o) based on the input (i), kernel (k), and stride (s).

Stride

In a Convolutional Neural Network (CNN), **stride** (s) is the distance between two consecutive positions of the kernel (or pooling window) as it traverses an input feature map along a specific axis.

Description and Mechanics

- **The "Step" Size:** While a unit stride ($s=1$) means the kernel moves one pixel at a time, a stride greater than one means the kernel "hops" over pixels.
- **Subsampling:** Mathematically, using a stride $s > 1$ is equivalent to performing a convolution with a



stride of 1 and then retaining only every *s*-th output element. This acts as a form of spatial subsampling.

- **Output Dimension:** Stride is a key denominator in the formula for calculating output size (*o)*. Because it is in the denominator, increasing the stride significantly reduces the spatial resolution of the resulting feature map.
- **Receptive Field Multiplier:** Strides in early layers have a multiplicative effect on the receptive field of all subsequent layers. The "effective stride" at the input (*S_0*) is the product of the strides of all preceding layers.

Typical Applications

1. **Dimensionality Reduction:** Strided convolutions are used to reduce the spatial resolution of feature maps. This decreases the number of operations and memory required for subsequent layers while allowing the network to become "deeper" and more abstract.
2. **Expanding Context (Receptive Field):** Using strides is an efficient way to quickly increase the receptive field size (*r_0*). This allows deeper neurons to "see" larger regions of the input image, which is essential for recognizing large objects or understanding global context.
3. **Overlapping vs. Non-Overlapping Pooling:** In pooling layers, if the stride (*s*) is equal to the window size (*z*), the regions are non-overlapping. If $s < z$, it results in overlapping pooling, which has been shown to reduce top-1 and top-5 error rates and make models more difficult to overfit.
4. **Upsampling (Fractional Strides):** In transposed convolutions (often used in autoencoders), "fractional" strides are conceptually used to map from smaller dimensions to larger ones. This is achieved by inserting zeros between input units to "stretch" the input, effectively allowing the kernel to move at a slower pace.
5. **Computational Efficiency:** Replicated convolutional networks (such as Space Displacement Neural Networks or SDNNs) can be scanned over large images very efficiently by skipping locations according to a stride, rather than recomputing overlapping areas.

Dilation

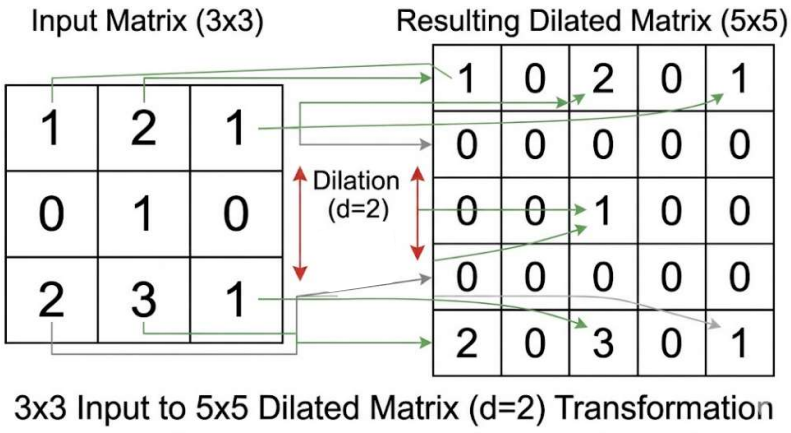
Dilation (often referred to as *atrous* convolution) is a technique in Convolutional Neural Networks that "inflates" the kernel by inserting spaces between its elements. This allows the network to cover a larger spatial area of the input without increasing the number of trainable parameters or the computational cost of the multiplication.

1. Mechanics and Effective Kernel Size

The spacing between kernel weights is controlled by a hyperparameter called the **dilation rate (*d*)**.

- **Standard Convolution (*d=1*):** Weights are applied to adjacent pixels.
- **Dilated Convolution (*d > 1*):** There are *d-1* spaces inserted between each weight.
- **Effective Kernel Size ($\hat{k}$):** Because of these gaps, a kernel of size *k* covers a wider span on the input map, calculated as:

$$\hat{k} = \mathbf{d}(\mathbf{k}-\mathbf{1})+\mathbf{1} \quad \text{or} \quad \hat{k} = \mathbf{k} + (\mathbf{k}-\mathbf{1})(\boldsymbol{d}-\mathbf{1})$$



3x3 Input to 5x5 Dilated Matrix (d=2) Transformation

2. Key Applications in CNNs

- **Exponential Receptive Field Expansion:** By stacking layers with exponentially increasing dilation rates (e.g., *d=1, 2, 4, 8...*), the **receptive field grows exponentially** while the number of parameters remains linear. This is far more efficient than stacking many standard convolutional layers to achieve the same "field of view".
- **Resolution Preservation in Dense Prediction:** In tasks like semantic segmentation, traditional CNNs use pooling and striding to increase context, which unfortunately reduces the spatial resolution of the output. Dilated convolutions allow a unit to "see" a wider context while maintaining a full-resolution output.
- **Multi-Scale Context Aggregation:** Dilation is used to systematically aggregate information from multiple scales, which is critical for identifying objects that may appear at various sizes in an image.

3. Adjusted Output Equation

When calculating the spatial dimensions of an output feature map (***o***) for a dilated layer, the effective kernel size **($\hat{k}$)** must be substituted into the standard arithmetic formula:

$$o = \left\lfloor \frac{i+2p-[d(k-1)+1]}{s} \right\rfloor + 1 \text{ or } o = \left\lfloor \frac{i+2p-k-(k-1)(d-1)]}{s} \right\rfloor + 1$$

Where ***i*** is input size, ***p*** is padding, and ***s*** is stride. This ensures designers can precisely control the resolution of feature maps even when expanding the field of view.

Receptive field

In Convolutional Neural Networks (CNNs), the concepts of Receptive Field (RF) and Effective Receptive Field (ERF) describe the spatial relationship between input data and the network's features, while context refers to the broader information required for accurate prediction.

The Receptive Field (RF)

The **receptive field** (or field of view) is the specific region of the input image that influences the value of a particular unit in a network layer.

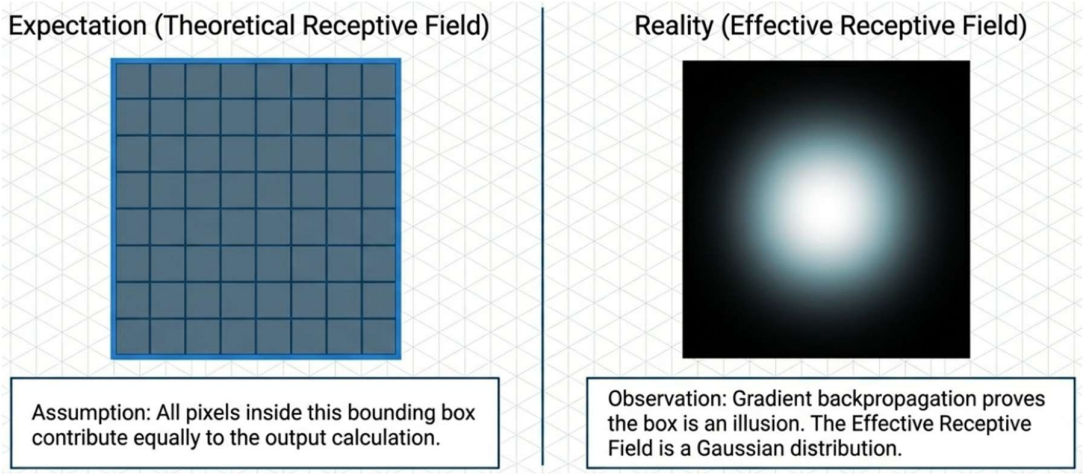
- **Local Connectivity:** Unlike standard Artificial Neural Networks (ANNs) where every unit depends on the entire input, CNN units have local receptive fields. This allows neurons to extract elementary visual features like edges, corners, and endpoints.
- **Hierarchical Growth:** As you go deeper into the network, the RF size increases. Features in subsequent layers combine local features from previous layers to detect higher-order, more abstract patterns.
- **Arithmetic Growth:** The theoretical RF size grows linearly as more layers are stacked (each layer adds ***k-1*** to the size) and multiplicatively through subsampling or striding. The closed-form expression for the RF size at the input (*r_0*) after *L* layers is:

$$\mathbf{r_0} = \sum_{l-1}^{L} \left((\mathbf{k_l} - \mathbf{1}) \prod_{i=1}^{l-1} \mathbf{s_i} \right) + \mathbf{1}$$

The Effective Receptive Field (ERF)

While the theoretical RF covers a specific span, research shows that not all pixels within that span contribute equally to the output. The Effective Receptive Field is the central region where input pixels have a non-negligible impact on a unit.

- **Gaussian Distribution:** The impact of pixels within an RF follows an asymptotically Gaussian distribution, peaking at the center and decaying rapidly toward the edges. This occurs because central pixels have significantly more "paths" to propagate information to the output unit than those at the periphery.
- **Relative Shrinkage:** Surprisingly, the ERF only occupies a small fraction of the full theoretical RF. While the theoretical RF grows at *O(n)*, the ERF grows more slowly at a rate of $O(\sqrt{n})$.



- **Architectural Impacts:** Subsampling and dilated convolutions significantly increase the ERF, whereas **skip-connections** (like those in ResNet) actually make the ERF smaller. Interestingly, the ERF has been observed to expand during the training process.

Worked Example

The following section describes the basic mathematical computations for an assumed input matrix of 5x5 and a kernel size of 2x2. Stride of '1' and a dilation factor of 2.

Input Matrix (5x5)

6	3	7	4	6
9	2	6	7	4
3	7	7	2	5
4	1	7	5	1
4	0	9	5	8

Kernel (2x2)

-2	0
0	-1

Parameters:

- Stride (S): 1
- Padding (P): 1
- Dilation (D): 2

Effective Kernel Size (Dilation)

With a dilation of 2, the kernel weights are spread across a larger footprint.

$$K_{eff} = K + (K - 1)(D - 1)$$
$$K_{eff} = 2 + (2 - 1)(2 - 1) = 3$$

The kernel behaves as a 3×3 filter where only the corners are active.

Output Dimensions

$$O = \lfloor (I + 2P - K_{eff}) / S \rfloor + 1$$
$$O = \lfloor (5 + 2(1) - 3) / 1 \rfloor + 1 = 5$$

The resulting feature map will be a 5×5 matrix.

Walkthrough:

The input is padded with zeros (P=1). The dilated kernel extracts values at the corners of a 3x3 window.

Step 1 (Position 0,0): The window starts at the top-left of the padded input. Corners: (0,0), (0,2), (2,0), (2,2).

$$(-2 \times 0) + (0 \times 0) + (0 \times 0) + (-1 \times 2) = -2$$

Step 2 (Position 0,1): Shift right by 1 stride. Corners: (0,1), (0,3), (2,1), (2,3).

$$(-2 \times 0) + (0 \times 0) + (0 \times 9) + (-1 \times 6) = -6$$

Step 3 (Position 0,2): Shift right by 1 stride. Corners: (0,2), (0,4), (2,2), (2,4).

$$(-2 \times 0) + (0 \times 0) + (0 \times 2) + (-1 \times 7) = -7$$

Completing the convolution across the entire grid yields the following Output Matrix (O):

-2	-6	-7	-4	0
-7	-19	-8	-19	-8
-1	-25	-9	-13	-14
0	-15	-19	-22	-4
0	-8	-2	-14	-10

Refer to the provided Jupyter notebook "***walkthrough.ipynb***" for a step by step guide of the above-mentioned computation.

Reference Papers

Gradient-Based Learning Applied to Document Recognition [1]

Y. Lecun, L. Bottou, Y. Bengio, and P. Haffner

This milestone paper from 1998 advocates replacing hand-designed heuristics with automatic gradient-based learning for pattern recognition. It highlights Convolutional Neural Networks (CNNs), LeNet-5, which utilize local receptive fields and weight sharing to achieve shift and distortion invariance. Trained via backpropagation, CNNs outperform traditional methods on the MNIST benchmark. The authors also introduce Graph Transformer Networks (GTNs) to enable the global training of multimodule systems. Successfully deployed for commercial check recognition, this paradigm minimizes global error by allowing modules like segmentation and recognition to cooperate.

ImageNet Classification with Deep Convolutional Neural Networks [2]

A. Krizhevsky, I. Sutskever, and G. E. Hinton

The authors developed a deep convolutional neural network with 60 million parameters to classify 1.2 million ImageNet images into 1000 categories. Achieving record-breaking error rates, the architecture features five convolutional and three fully-connected layers. Key innovations include Rectified Linear Units (ReLUs) for faster training and "dropout" to prevent overfitting. The model utilizes an optimized implementation across two GPUs to manage its massive size. Other enhancements, such as local response normalization and overlapping pooling, further improved generalization performance, enabling the model to dominate the ILSVRC competitions.

Multi-scale context aggregation by dilated convolutions [3]

F. Yu and V. Koltun

The authors develop a convolutional network module specifically for dense prediction tasks like semantic segmentation. This module utilizes dilated convolutions to systematically aggregate multi-scale contextual information without losing spatial resolution or coverage. By applying filters with exponentially increasing dilation factors, the architecture achieves an exponential expansion of the receptive field while parameters grow only linearly. They also propose a simplified "front-end" module by removing pooling and striding from standard classification networks. This approach reliably increases accuracy across various state-of-the-art architectures, including FCN and DeepLab, on the Pascal VOC benchmark.

A guide to convolution arithmetic for deep learning [4]

Vincent Dumoulin and Francesco Visin provide a comprehensive technical guide to convolution arithmetic, detailing the mathematical relationships between input/output shapes and hyperparameters like kernel size, stride, and padding. The paper systematically derives equations for standard convolutional and pooling layers, extending this framework to transposed convolutions (often called "deconvolutions"). It clarifies transposed operations by framing them as direct convolutions on "stretched" inputs using fractional strides. Additionally, the authors explain dilated convolutions, which cheaply expand the receptive field without increasing parameters. This implementation-independent resource serves as a foundational spatial tutorial.

Understanding the Effective Receptive Field in Deep Convolutional Neural Networks [5]

Luo et al. introduce the Effective Receptive Field (ERF), defining it as the region containing input pixels with a non-negligible impact on an output unit. They demonstrate that the distribution of this impact is asymptotically Gaussian, peaking at the centre and decaying rapidly toward the

edges. Consequently, the ERF occupies only a small fraction of the theoretical receptive field and grows at a slower rate. The authors show that sub-sampling and dilation expand the ERF, while skip-connections shrink it. Finally, they observe that the ERF expands during training.

Computing Receptive Fields of Convolutional Neural Networks [6]

Araujo et al. provide a mathematical framework and an open-source library to automate the computation of receptive fields in CNNs. They derive a closed-form expression for single-path architectures and extend this to arbitrary computation graphs with multiple paths, such as ResNet. The paper details alignment criteria to ensure output features are cantered consistently across different network paths. Their research identifies a logarithmic relationship between classification accuracy and receptive field size, highlighting that while expansive fields improve high-level recognition, they provide diminishing rewards.

[1] Y. Lecun, L. Bottou, Y. Bengio, and P. Haffner, "Gradient-based learning applied to document recognition," *Proceedings of the IEEE*, vol. 86, no. 11, pp. 2278–2324, Nov. 1998, doi: 10.1109/5.726791.
[url]: https://ieeexplore.ieee.org/abstract/document/726791

[2] A. Krizhevsky, I. Sutskever, and G. E. Hinton, "ImageNet classification with deep convolutional neural networks," *Commun. ACM*, vol. 60, no. 6, pp. 84–90, May 2017, doi: 10.1145/3065386.
[url]: https://dl.acm.org/doi/10.1145/3065386

[3] F. Yu and V. Koltun, "Multi-Scale Context Aggregation by Dilated Convolutions," Apr. 30, 2016, *arXiv*: arXiv:1511.07122. doi: 10.48550/arXiv.1511.07122.
[url]: http://arxiv.org/abs/1511.07122

[4] V. Dumoulin and F. Visin, "A guide to convolution arithmetic for deep learning," Jan. 11, 2018, *arXiv*: arXiv:1603.07285. doi: 10.48550/arXiv.1603.07285.
[url]: http://arxiv.org/abs/1603.07285

[5] W. Luo, Y. Li, R. Urtasun, and R. Zemel, "Understanding the Effective Receptive Field in Deep Convolutional Neural Networks," Jan. 25, 2017, *arXiv*: arXiv:1701.04128. doi: 10.48550/arXiv.1701.04128.
[url]: http://arxiv.org/abs/1701.04128

[6] A. Araujo, W. Norris, and J. Sim, "Computing Receptive Fields of Convolutional Neural Networks," *Distill*, vol. 4, no. 11, p. e21, Nov. 2019, doi: 10.23915/distill.00021.
[url]: https://distill.pub/2019/computing-receptive-fields